your application. You might be willing to release software that has a high bug count if all the bugs are of a low severity.

The inconsistency with bugs is that *severity* is also a relative term. To a designer the wrong font in the footer may be a critical bug, but to a developer concerned with functionality, maybe it shouldn't even be a bug in the first place. In this case, severity is in the eye of the beholder.

The charts and graphs you've seen up to this point are all useful in their own right even though they measure relative data. To go beyond the standard PTS graphs there are some best practices you can follow to ensure you have data that's enriched for analysis and ultimately for combination with other data sources.

3.2. Prepare for analysis; generate the richest set of data you can

As you collect data, keep in mind that your analysis will only be as good as the data you collect; how your team uses your PTS will translate into what you can glean from the data in the system. To set yourself and your team up for success by getting the most meaningful analysis from this data, this section offers tips on working with your PTS to help ensure you're generating as much useful data as possible from it.

Most APIs will give you something like a list of work items based on the query you make, and each work item will have many pieces of data attached to it. The data you get back will give you information such as:

- Who is working on what
- When things get done in a good way

Those two statements are general and loaded with hidden implications. We break them down in figure 3.6.

Figure 3.6. Parsing the loaded words that tell you how PTS is used